AGENT SDK

Quickstart

Get started with the Python or TypeScript Agent SDK to build AI agents that work autonomously

Use the Agent SDK to build an AI agent that reads your code, finds bugs, and fixes them, all without manual intervention.

What you'll do:

1. Set up a project with the Agent SDK
2. Create a file with some buggy code
3. Run an agent that finds and fixes the bugs automatically

Prerequisites

- **Node.js 18+** or **Python 3.10+**
- An **Anthropic account** (sign up here)

Setup

1 Create a project folder

Create a new directory for this quickstart:

```
mkdir my-agent
cd my-agent
```

For your own projects, you can run the SDK from any folder; it will have access to files in that directory and its subdirectories by default.

2 Install the SDK

Install the Agent SDK package for your language:

TypeScript (new project)

```
npm init -y
npm pkg set type=module
npm install @anthropic-ai/claude-agent-sdk
npm install --save-dev tsx
```

Setting `"type": "module"` in `package.json` lets your agent script use top-level `await`, and **tsx** runs TypeScript files directly.

TypeScript (existing project)

```
npm install @anthropic-ai/claude-agent-sdk
npm install --save-dev tsx
```

tsx runs TypeScript files directly. If your project uses CommonJS, name your agent script `agent.mts` instead of `agent.ts`. The `.mts` extension makes tsx treat the file as an ES module, so top-level `await` works without converting your whole project to ES modules. Use `agent.mts` in place of `agent.ts` in the create and run steps later in this quickstart.

Python (uv)

uv is a fast Python package manager that handles virtual environments automatically:

```
uv init
uv add claude-agent-sdk
```

Python (pip)

Create and activate a virtual environment, then install the package.On macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install claude-agent-sdk
```

On Windows:

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install claude-agent-sdk
```

If PowerShell blocks `Activate.ps1` with an execution policy error, run `Set-ExecutionPolicy -Scope Process RemoteSigned` first.

> The TypeScript SDK bundles a native Claude Code binary for your platform as an optional dependency, so you don't need to install Claude Code separately.

3 Set your API key

Get an API key from the **Claude Console**, then set it as an environment variable in the shell where you'll run your agent:

macOS / Linux

```
export ANTHROPIC_API_KEY=your-api-key
```

Windows (PowerShell)

```
$env:ANTHROPIC_API_KEY = "your-api-key"
```

The SDK reads the key from the environment of the process that runs your agent; it doesn't

load `.env` files automatically. If you keep the key in a `.env` file, load it yourself, for example with the `dotenv` package, before calling the SDK.

The SDK also supports authentication via third-party API providers:

- **Amazon Bedrock**: set `CLAUDE_CODE_USE_BEDROCK=1` environment variable and configure AWS credentials
- **Claude Platform on AWS**: set `CLAUDE_CODE_USE_ANTHROPIC_AWS=1` and `ANTHROPIC_AWS_WORKSPACE_ID`, then configure AWS credentials
- **Google Vertex AI**: set `CLAUDE_CODE_USE_VERTEX=1` environment variable and configure Google Cloud credentials
- **Microsoft Azure**: set `CLAUDE_CODE_USE_FOUNDRY=1` environment variable and configure Azure credentials

See the setup guides for **Bedrock**, **Claude Platform on AWS**, **Vertex AI**, or **Azure AI Foundry** for details.

> Unless previously approved, Anthropic does not allow third party developers to offer claude.ai login or rate limits for their products, including agents built on the Claude Agent SDK. Please use the API key authentication methods described in this document instead.

Create a buggy file

This quickstart walks you through building an agent that can find and fix bugs in code. First, you need a file with some intentional bugs for the agent to fix. Create `utils.py` in the `my-agent` directory and paste the following code:

```
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total += num
    return total / len(numbers)


def get_user_name(user):
    return user["name"].upper()
```

This code has two bugs:

1. `calculate_average([])` crashes with division by zero
2. `get_user_name(None)` crashes with a TypeError

Build an agent that finds and fixes bugs

Create `agent.py` if you're using the Python SDK, or `agent.ts` for TypeScript. Use `agent.mts` instead if your existing project uses CommonJS:

This code has three main parts:

1. `query`: the main entry point that creates the agentic loop. It returns an async iterator, so you use `async for` to stream messages as Claude works. See the full API in the **Python** or **TypeScript** SDK reference.
2. `prompt`: what you want Claude to do. Claude figures out which tools to use based on the task.
3. `options`: configuration for the agent. This example uses `allowedTools` to pre-approve `Read`, `Edit`, and `Glob`, and `permissionMode: "acceptEdits"` to auto-approve file changes. Other options include `systemPrompt`, `mcpServers`, and more. See all options for **Python** or **TypeScript**.

The `async for` loop keeps running as Claude thinks, calls tools, observes results, and decides what to do next. Each iteration yields a message: Claude's reasoning, a tool call, a tool result, or the final outcome. The SDK handles the orchestration (tool execution, context management, retries) so you just consume the stream. The loop ends when Claude finishes the task or hits an error.

The message handling inside the loop filters for human-readable output. Without filtering, you'd see raw message objects including system initialization and internal state, which is useful for debugging but noisy otherwise.

> This example uses streaming to show progress in real-time. If you don't need live output (e.g., for background jobs or CI pipelines), you can collect all messages at once. See **Streaming vs. single-turn mode** for details.

Run your agent

Your agent is ready. Run it with the following command:

TypeScript

```
npm init -y
npm pkg set type=module
npm install @anthropic-ai/claude-agent-sdk
npm install --save-dev tsx
```

Setting `"type": "module"` in `package.json` lets your agent script use top-level `await`, and **tsx** runs TypeScript files directly.

Python (uv)

```
npm install @anthropic-ai/claude-agent-sdk
npm install --save-dev tsx
```

tsx runs TypeScript files directly. If your project uses CommonJS, name your agent script `agent.mts` instead of `agent.ts`. The `.mts` extension makes tsx treat the file as an ES module, so top-level `await` works without converting your whole project to ES modules. Use `agent.mts` in place of `agent.ts` in the create and run steps later in this quickstart.

Python (pip)

uv is a fast Python package manager that handles virtual environments automatically:

```
uv init
uv add claude-agent-sdk
```

Create and activate a virtual environment, then install the package.On macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install claude-agent-sdk
```

On Windows:

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install claude-agent-sdk
```

If PowerShell blocks `Activate.ps1` with an execution policy error, run `Set-ExecutionPolicy -Scope Process RemoteSigned` first.

```
export ANTHROPIC_API_KEY=your-api-key
```

```
$env:ANTHROPIC_API_KEY = "your-api-key"
```

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage,
ResultMessage


async def main():
    # Agentic loop: streams messages as Claude works
    async for message in query(
        prompt="Review utils.py for bugs that would cause crashes. Fix any issues you
find.",
```

```
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Glob"],  # Auto-approve these tools
            permission_mode="acceptEdits",  # Auto-approve file edits
        ),
    ):
        # Print human-readable output
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if hasattr(block, "text"):
                    print(block.text)  # Claude's reasoning
                elif hasattr(block, "name"):
                    print(f"Tool: {block.name}")  # Tool being called
        elif isinstance(message, ResultMessage):
            print(f"Done: {message.subtype}")  # Final result


asyncio.run(main())
```

```
npx tsx agent.ts
```

If you named your script `agent.mts`, run `npx tsx agent.mts` instead.

```
uv run agent.py
```

With your virtual environment still activated:

```
python agent.py
```

```
options = ClaudeAgentOptions(
    allowed_tools=["Read", "Edit", "Glob", "WebSearch"],
permission_mode="acceptEdits"
)
```

```
options = ClaudeAgentOptions(
    allowed_tools=["Read", "Edit", "Glob"],
    permission_mode="acceptEdits",
    system_prompt="You are a senior Python developer. Always follow PEP 8 style
guidelines.",
)
```

```
options = ClaudeAgentOptions(
    allowed_tools=["Read", "Edit", "Glob", "Bash"], permission_mode="acceptEdits"
)
```

As it works, the agent prints its reasoning and each tool it calls, ending with `Done: success`. After running, check `utils.py`. You'll see defensive code handling empty lists and null users. Your agent autonomously:

1. **Read** `utils.py` to understand the code
2. **Analyzed** the logic and identified edge cases that would crash
3. **Edited** the file to add proper error handling

This is what makes the Agent SDK different: Claude executes tools directly instead of asking you to implement them.

> If you see "API key not found", make sure you've set the `ANTHROPIC_API_KEY` environment variable in the shell where you run your agent. The SDK doesn't load `.env` files automatically. See the **full troubleshooting guide** for more help.

Try other prompts

Now that your agent is set up, try some different prompts:

- `"Add docstrings to all functions in utils.py"`
- `"Add type hints to all functions in utils.py"`
- `"Create a README.md documenting the functions in utils.py"`

Customize your agent

You can modify your agent's behavior by changing the options. Here are a few examples:

Add web search capability:

Give Claude a custom system prompt:

Run commands in the terminal:

With `Bash` enabled, try: `"Write unit tests for utils.py, run them, and fix any failures"`

Key concepts

Tools control what your agent can do:

Tools	What the agent can do
`Read` , `Glob` , `Grep`	Read-only analysis
`Read` , `Edit` , `Glob`	Analyze and modify code
`Read` , `Edit` , `Bash` , `Glob` , `Grep`	Full automation

Permission modes control how much human oversight you want:

Mode	Behavior	Use case
`acceptEdits`	Auto-approves file edits and common filesystem commands, asks for other actions	Trusted development workflows
`plan`	Runs read-only tools; file edits are never auto-approved and reach your `canUseTool` callback	Scoping a task before approving execution
`dontAsk`	Denies anything not in `allowedTools`	Locked-down headless agents
`auto` (TypeScript only)	A model classifier approves or denies each tool call	Autonomous agents with safety guardrails
`bypassPermissions`	Runs every tool without prompting, unless an explicit `ask` **rule** matches	Sandboxed CI, fully trusted environments
`default`	Requires a `canUseTool` callback to handle approval	Custom approval flows

The example above uses `acceptEdits` mode, which auto-approves file operations so the agent can

run without interactive prompts. If you want to prompt users for approval, use `default` mode and provide a `canUseTool` **callback** that collects user input. For more control, see **Permissions**.

Next steps

Now that you've created your first agent, learn how to extend its capabilities and tailor it to your use case:

- **Permissions**: control what your agent can do and when it needs approval
- **Hooks**: run custom code before or after tool calls
- **Sessions**: build multi-turn agents that maintain context
- **MCP servers**: connect to databases, browsers, APIs, and other external systems
- **Hosting**: deploy agents to Docker, cloud, and CI/CD
- **Example agents**: see complete examples: email assistant, research agent, and more